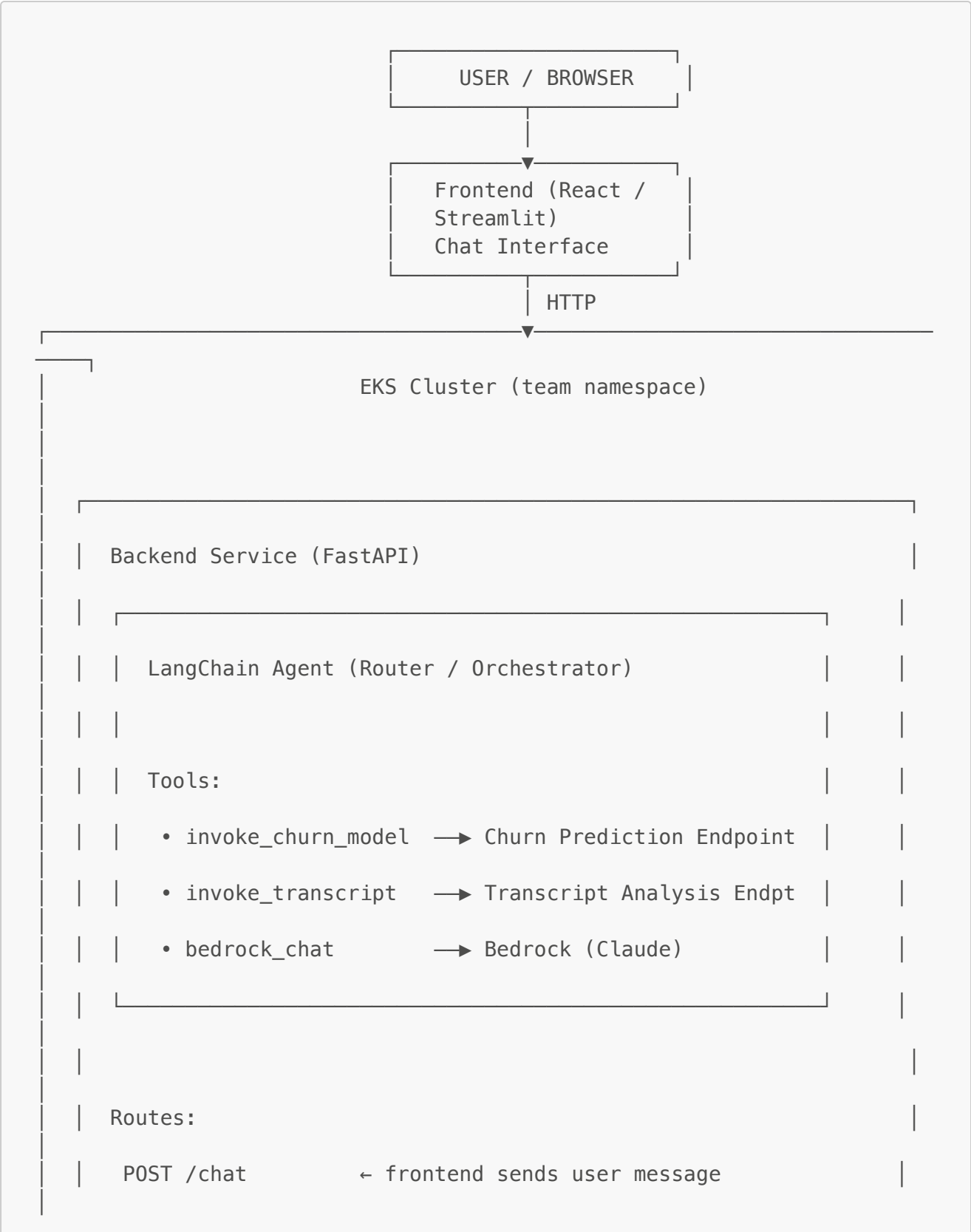
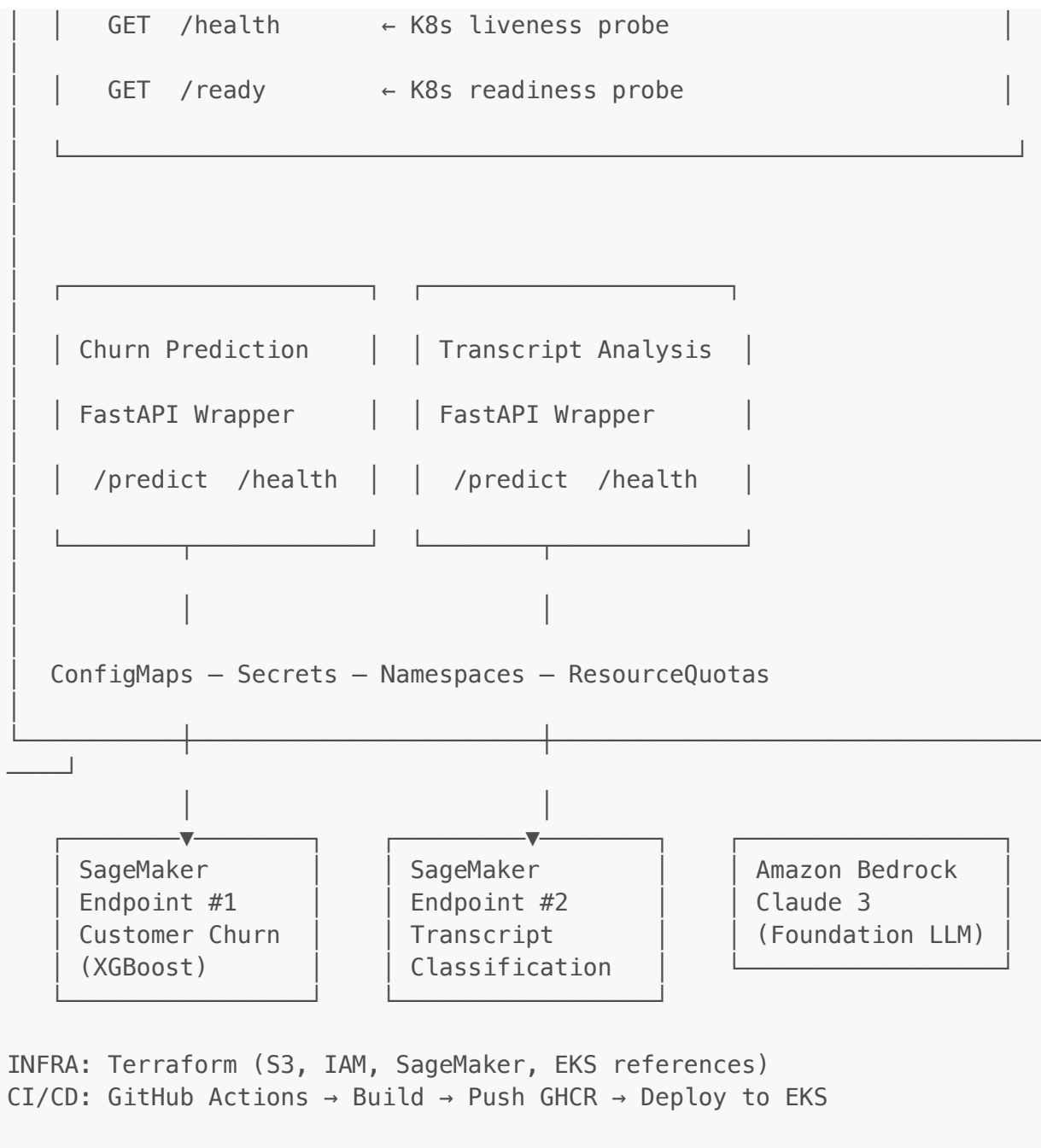


The Retention Engine — Project Roadmap

Team: Troy, Kathleen, Okino, George

Platform Architecture





Repo Structure

```
capstone/
├── .github/
│   └── workflows/
│       ├── ci-backend.yaml           # Build + push backend image
│       ├── ci-frontend.yaml         # Build + push frontend image
│       ├── ci-ml-wrappers.yaml      # Build + push ML wrapper images
│       ├── deploy.yaml              # kubectl apply to EKS
│       └── terraform.yaml           # (bonus) tf plan/apply
├── terraform/
│   ├── main.tf
│   ├── variables.tf
│   └── outputs.tf
```

```

├── providers.tf
├── modules/                                # (bonus) networking, iam, sagemaker
├── k8s/
│   ├── namespace.yaml
│   ├── configmaps/
│   │   ├── agent-config.yaml
│   ├── deployments/
│   │   ├── agent-deployment.yaml
│   │   ├── backend-deployment.yaml
│   │   ├── churn-wrapper-deployment.yaml
│   │   ├── frontend-deployment.yaml
│   │   ├── transcript-wrapper-deployment.yaml
│   ├── secrets.yaml                        # (template – real values via GH
Secrets)
│   ├── services/
│   │   ├── backend-service.yaml
│   │   ├── churn-wrapper-service.yaml
│   │   ├── transcript-wrapper-service.yaml
│   │   ├── frontend-service.yaml
│   ├── backend/
│   │   ├── Dockerfile
│   │   ├── app/
│   │   │   ├── main.py                    # FastAPI app, /chat, /health,
/ready
│   │   │   ├── agent.py                    # LangChain agent + tool definitions
│   │   │   ├── tools/
│   │   │   │   ├── churn_tool.py          # Calls churn-wrapper /predict
│   │   │   │   ├── transcript_tool.py     # Calls transcript-wrapper /predict
│   │   │   │   ├── bedrock_tool.py       # Bedrock invoke for general chat
│   │   │   └── config.py                  # Env var config
│   └── requirements.txt
├── ml-wrappers/
│   ├── churn-predictor-api/
│   │   ├── Dockerfile
│   │   ├── app.py                         # FastAPI /predict + /health
│   │   └── requirements.txt
│   ├── sentiment-analysis-api/
│   │   ├── Dockerfile
│   │   ├── app.py                         # FastAPI /predict + /health
│   │   └── requirements.txt
├── sagemaker/
│   ├── churn/
│   │   ├── train.py                       # Training script or notebook
│   │   ├── deploy.py                      # Endpoint creation script
│   │   └── data/                          # Sample data / data prep scripts
│   ├── sentiment/
│   │   ├── sentiment_training.ipynb
│   │   ├── deploy.py
│   │   └── data/
├── frontend/
│   ├── Dockerfile
│   ├── src/                              # React or Streamlit app
└── ...

```

```

├── docker-compose.yaml           # Local dev: all services together
├── docs/
│   ├── architecture.md
│   ├── setup.md
│   ├── deployment.md
│   ├── teardown.md
│   └── llm-usage/               # Per-member LLM usage docs
├── PROJECT_ROADMAP.md
└── README.md

```

Owner Matrix

Component	Primary Owner	Support	Key Deliverables
GitHub Projects / Kanban	Troy	All	Board setup, task cards, sprint tracking
GitHub Actions CI/CD	Troy	George	All .yaml workflows, secrets config
Dataset identification	Kathleen + Okino	—	Churn dataset, transcript dataset
SageMaker training + deploy	Kathleen + Okino	—	sagemaker/ training scripts, live endpoints
ML FastAPI wrappers	Kathleen + Okino	Troy (CI)	ml-wrappers/ services with /predict + /health
Terraform infra	George	Troy (CI)	terraform/ — IAM, S3, SageMaker resources
Kubernetes manifests	George	Troy (deploy wf)	k8s/ — all manifests
LangChain agent + Bedrock	George	Kathleen/Okino (tool contracts)	backend/app/agent.py , tool definitions
Backend FastAPI service	George	All	backend/ — routes, agent wiring
Frontend chat UI	All	—	frontend/ — chat interface
Documentation + diagrams	All	—	docs/ , README.md

Graded Areas → Owner Mapping

#	Graded Area (10% each)	Primary Owner	Supporting
1	Containers & Dockerfiles	George (backend, frontend), Kathleen/Okino (ML wrappers)	Troy (CI builds)
2	Terraform Provisioning	George	—
3	CI/CD with GitHub Actions	Troy	All contribute workflows
4	LangChain Agentic Harness	George	Kathleen, Okino (tool contracts)
5	Bedrock Chat Agent & Frontend	George (Bedrock/agent), All (frontend)	—
6	SageMaker ML Endpoints	Okino (Endpoint 1: QA/Transcript), Kathleen (Endpoint 2: Churn)	George (integration)
7	Kubernetes Orchestration	George	Troy (CI/CD deploy steps)
8	Team Collaboration & Agile	Troy (board setup), All (participation)	—
9	Presentation & Documentation	All	—
10	Leveraging LLMs as Dev Tools	All (individual docs)	—

Interface Contracts

Define these early so everyone can work in parallel.

Churn Wrapper → SageMaker Endpoint

```
POST /predict
Request: { "features": [0.5, 1.0, 3, 45.2, ...] }
Response: { "churn_probability": 0.82, "prediction": "churn" }

GET /health
Response: { "status": "healthy", "endpoint": "churn-endpoint-v1" }
```

Transcript Wrapper → SageMaker Endpoint

```
POST /predict
Request: { "text": "I've been waiting for 30 minutes and nobody..." }
Response: { "category": "complaint", "sentiment": "negative",
"confidence": 0.91 }

GET /health
Response: { "status": "healthy", "endpoint": "transcript-endpoint-v1" }
```

Backend /chat → Frontend

```
POST /chat
Request: { "message": "What's the churn risk for customer 4521?" }
Response: { "response": "Based on the model prediction, customer 4521
has a 82% churn probability..." }

GET /health
Response: { "status": "healthy", "agent": "ready", "tools": ["churn",
"transcript", "bedrock"] }
```

LangChain Agent Design

```
# Simplified agent structure – George owns, Kathleen/Okino define tool
I/O

tools = [
    # Tool 1: Churn prediction
    #   Input: customer feature vector
    #   Action: POST to churn-wrapper-service:8000/predict
    #   Output: churn probability + label

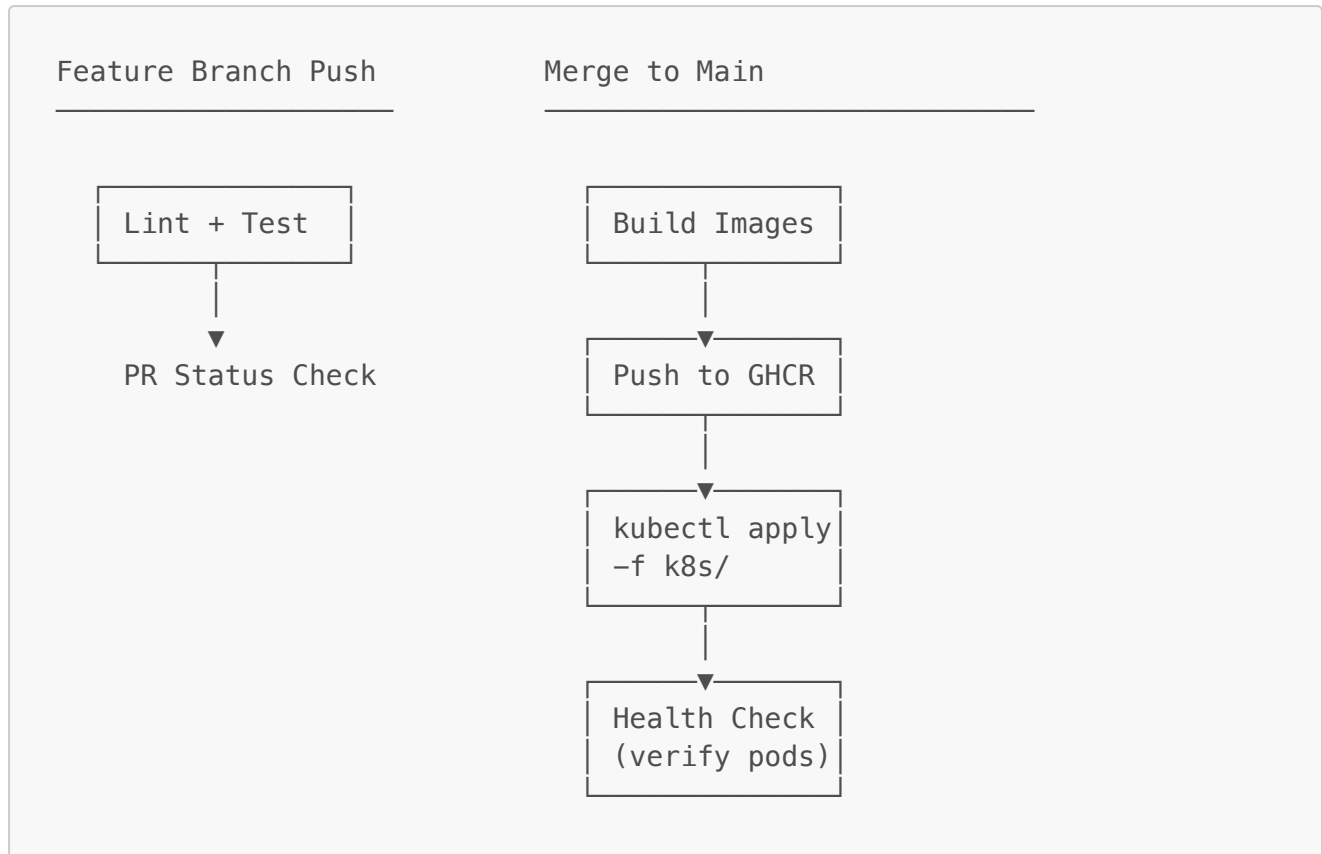
    # Tool 2: Transcript analysis
    #   Input: raw text from service call
    #   Action: POST to transcript-wrapper-service:8000/predict
    #   Output: category + sentiment + confidence

    # Tool 3: Bedrock general chat
    #   Input: user message (when no tool needed)
    #   Action: bedrock invoke_model (Claude 3)
    #   Output: conversational response
]

# Agent routes based on user intent:
#   "Will customer X churn?"      → churn tool
#   "Analyze this transcript..." → transcript tool
#   "What can you help with?"     → bedrock general chat
#   Complex queries               → chain multiple tools
```

```
# Non-trivial agentic behavior:
#   If churn_probability > 0.7 → auto-generate retention offer via
Bedrock
#   Multi-step: transcript → QA score → combine with account data →
churn → retention offer
```

CI/CD Pipeline Overview



Workflows (Troy owns):

File	Trigger	What It Does
ci-backend.yaml	push to backend/**	Build + push backend image
ci-ml-wrappers.yaml	push to ml-wrappers/**	Build + push both wrapper images
ci-frontend.yaml	push to frontend/**	Build + push frontend image
deploy.yaml	merge to main	kubectl apply -f k8s/ + health check
terraform.yaml	(bonus) manual trigger	tf plan on PR, tf apply on merge

Kubernetes Layout

Namespace: team-retention-engine

— backend-deployment	(2 replicas)
└ backend-service	(ClusterIP or LoadBalancer)
— churn-wrapper-deployment	(1 replica)
└ churn-wrapper-service	(ClusterIP)
— transcript-wrapper-deployment	(1 replica)
└ transcript-wrapper-service	(ClusterIP)
— frontend-deployment	(1 replica)
└ frontend-service	(LoadBalancer → public)
— configmap	(endpoint URLs, region, model names)
— secret	(AWS creds – injected via GH Actions)

All inter-service calls stay **inside the cluster** (ClusterIP). Only the frontend service gets a LoadBalancer for public access. The backend talks to ML wrappers via K8s DNS (`churn-wrapper-service.team-retention-engine.svc.cluster.local:8000`).

Phase Overview

Phase 0: Setup & Scaffolding		(Days 1–2)
Phase 1: Core Services		(Days 3–5)
Phase 2: Infra, Deploy & CI/CD		(Days 5–7)
Phase 3: Frontend, Polish & Docs		(Days 7–9)

Phase 0 — Setup & Scaffolding (Days 1–2)

Goal: Repo structure, Kanban board, API contracts agreed, one vertical slice working locally.

Troy — Git & CI/CD Setup

- ☐ Set up GitHub Projects Kanban board with columns: **Backlog**, **In Progress**, **In Review**, **Done**
- ☐ Create tasks from this roadmap as GitHub Issues and assign owners
- ☐ Set branch protection on **main** (require PR + 1 approval)
- ☐ Agree on branch naming convention with team (e.g., **feat/<owner>-<description>**, **fix/...**)
- ☐ Scaffold **.github/workflows/** with placeholder workflow files
- ☐ Create **docker-compose.yaml** skeleton for local dev

George — Infrastructure & Backend Scaffold

- ☐ Scaffold **terraform/** directory (main.tf, variables.tf, outputs.tf, providers.tf)
- ☐ Identify which AWS resources are pre-existing (shared VPC, EKS cluster, IAM roles) vs. need provisioning
- ☐ Scaffold **k8s/** with namespace + configmap templates
- ☐ Scaffold **backend/** with FastAPI hello world (**/health**, **/ready**, **/chat** stubs)
- ☐ Write first Dockerfile for backend
- ☐ Verify Bedrock model access in **us-east-1**

Okino — ML Endpoint 1 (Transcript/QA Evaluator)

- ☐ Download sentiment dataset from Kaggle ("Customer Support on Twitter" or "Amazon Reviews")
- ☐ Scaffold **sagemaker/transcript/** with train.py notebook + data/ directory
- ☐ Scaffold **ml-wrappers/transcript/** with FastAPI stub (app.py, Dockerfile, requirements.txt)
- ☐ Begin data exploration and preprocessing

Kathleen — ML Endpoint 2 (Churn Predictor)

- ☐ Download "Telco Customer Churn" (IBM) dataset from Kaggle
- ☐ Scaffold **sagemaker/churn/** with train.py notebook + data/ directory
- ☐ Scaffold **ml-wrappers/churn/** with FastAPI stub (app.py, Dockerfile, requirements.txt)
- ☐ Begin data exploration and preprocessing

All — Day 1 Sync

- ☐ Agree on API contracts (see Interface Contracts section above)
- ☐ Each member documents their LLM setup in **docs/llm-usage/**
- ☐ Decide frontend framework: React (more polished, bonus points) or Streamlit (faster to build)

Milestone ✓

Every team member can run **docker-compose up** and hit the backend **/health** endpoint.

Phase 1 — Core Services (Days 3–5)

Goal: ML endpoints live, agent wired with tools, CI green. The core pipeline works locally.

Troy — CI/CD Pipelines

- ☐ CI workflow: build + push backend image to GHCR
- ☐ CI workflow: build + push ML wrapper images to GHCR
- ☐ CI workflow: build + push frontend image to GHCR
- ☐ Add verification step (health check) to each workflow
- ☐ Use **docker/build-push-action** + GitHub Secrets for registry credentials

Okino — QA/Transcript Evaluator (Train + Deploy)

- ☐ **Train Endpoint 1 in SageMaker:**
 - Preprocess sentiment dataset (tokenize, label encode)
 - Train DistilBERT/RoBERTa text classifier
 - Deploy to SageMaker endpoint
 - Verify: `aws sagemaker describe-endpoint --endpoint-name transcript-endpoint-v1`
- ☐ **FastAPI wrapper (ml-wrappers/transcript/):**
 - `POST /predict` — accepts `{"text": "..."}` , returns `{"category": "...", "sentiment": "...", "confidence": 0.91}`
 - `GET /health` — returns `{"status": "healthy", "endpoint": "transcript-endpoint-v1"}`
 - Invoke SageMaker endpoint from FastAPI using `boto3`
- ☐ Write Dockerfile (multi-stage build, slim base, `.dockerignore`, health check)
- ☐ Test: `curl localhost:8000/predict -d '{"text": "I am very upset"}'`

Kathleen — Churn Predictor (Train + Deploy)

- ☐ **Train Endpoint 2 in SageMaker:**
 - Preprocess Telco Churn dataset (encode categoricals, scale numerics)
 - Train XGBoost/LightGBM model
 - Deploy to SageMaker endpoint
 - Verify: `aws sagemaker describe-endpoint --endpoint-name churn-endpoint-v1`
- ☐ **FastAPI wrapper (ml-wrappers/churn/):**
 - `POST /predict` — accepts `{"features": [0.5, 1.0, 3, 45.2, ...]}`, returns `{"churn_probability": 0.82, "prediction": "churn"}`
 - `GET /health` — returns `{"status": "healthy", "endpoint": "churn-endpoint-v1"}`
 - Invoke SageMaker endpoint from FastAPI using `boto3`
- ☐ Write Dockerfile (multi-stage build, slim base, `.dockerignore`, health check)
- ☐ Test: `curl localhost:8000/predict -d '{"features": [...]}'`

George — LangChain Agent + Backend

- ☐ **Terraform apply** — provision core cloud resources:
 - S3 bucket (data storage)
 - IAM roles for SageMaker execution and Bedrock access
 - Reference existing VPC/EKS (use `data` sources or `terraform import`)
- ☐ Document lifecycle: `terraform init` → `plan` → `apply` in `docs/setup.md`
- ☐ Set up remote state with S3 + DynamoDB locking (bonus)
- ☐ **Write LangChain agent** with tool definitions:
 - `churn_tool` → calls churn-wrapper `/predict`
 - `transcript_tool` → calls transcript-wrapper `/predict`
 - `bedrock_tool` → Bedrock invoke for general chat
- ☐ Wire agent into backend `/chat` route
- ☐ Write K8s deployments + services for all containers

From local, you can POST to `/chat` with "Will this customer churn?" and the agent calls the churn endpoint and returns a response.

Phase 2 — Infrastructure, Deploy & CI/CD (Days 5–7)

Goal: Everything running on Kubernetes, CI/CD deploys automatically, full agentic pipeline working.

Troy — Deploy Pipeline

- ☐ Deploy workflow: `kubectl apply -f k8s/` triggered on merge to main
- ☐ Add rollback step to deploy workflow (bonus)
- ☐ Set up branch-based targeting: staging vs prod (bonus)
- ☐ Add verification steps:
 - Health check after deployment
 - `kubectl rollout status deployment/<name>`
 - Smoke test: `curl /health` endpoints

Kathleen & Okino — ML Hardening

- ☐ Create the Customer ID mapping logic between datasets
- ☐ Test cross-model pipeline: transcript → QA score → combine with account data → churn probability
- ☐ Harden ML wrappers (bonus):
 - Retry logic on SageMaker invocations
 - Error handling and fallback responses
 - Model versioning
- ☐ Test endpoints under load, verify CloudWatch logs

George — K8s Deploy & Agentic Pipeline

- ☐ Deploy full stack to Minikube/EKS: `kubectl apply -f k8s/`
- ☐ Add ConfigMaps + Secrets for env vars (endpoint URLs, AWS creds)
- ☐ Add health probes to all deployments (liveness + readiness)
- ☐ Verify Bedrock access from inside cluster
- ☐ **Full multi-step agentic workflow:**
 - Agent receives transcript → calls transcript tool → gets sentiment/QA score → calls churn tool with score + account data → if churn risk > 0.7, asks Bedrock to generate retention offer
 - This satisfies: "non-trivial agentic behavior" + "multi-step reasoning" + "tool use"
- ☐ Add Terraform resources for SageMaker endpoint configs, additional IAM policies
- ☐ Modularize Terraform (bonus): `modules/networking`, `modules/compute`, `modules/sagemaker`
- ☐ Add resource management (bonus): `ResourceQuota` + `LimitRange` per namespace
- ☐ LangGraph workflow (bonus): model pipeline as state graph with conditional edges
- ☐ LangSmith integration (bonus): `LANGCHAIN_TRACING_V2=true`

Milestone ✓

Push to **main** triggers build → push → deploy. All pods healthy in cluster. Chat works via LoadBalancer URL.

Phase 3 — Frontend, Polish & Docs (Days 7–9)

Goal: Chat UI live, full pipeline working, docs complete, demo-ready.

Frontend — Manager's Command Center (All)

- ☐ Chat interface for interacting with the agent
- ☐ Display: QA Score, Sentiment, Churn Probability, Retention Recommendation
- ☐ "High Risk" leaderboard or dashboard panel
- ☐ Conversation memory / context persistence (bonus)
- ☐ Polish with Tailwind/MUI (bonus)

Troy — Final CI/CD & Collaboration

- ☐ Wire frontend CI/CD
- ☐ Ensure all workflows pass on main
- ☐ Final Kanban cleanup: all cards resolved
- ☐ Write docs: CI/CD setup guide, teardown steps

Kathleen & Okino — ML Docs & Bonus

- ☐ Verify both SageMaker endpoints are stable and responding
- ☐ Ensure model invocations work from inside K8s pods
- ☐ Add 3rd endpoint or model versioning (bonus)
- ☐ Write docs: ML approach, data decisions, LLM usage log

George — Infrastructure Finalization

- ☐ Verify all Terraform state is clean and reproducible
- ☐ Test full lifecycle: **destroy** → **init** → **plan** → **apply**
- ☐ Ensure no hardcoded credentials anywhere
- ☐ Add HPA (Horizontal Pod Autoscaler) for high-traffic services (bonus)
- ☐ Test: **kubectl exec <pod> -- curl localhost:8000/health** for each service
- ☐ Write docs: architecture diagram, infra setup, deployment steps, teardown

Bonus Integrations (Anyone)

- ☐ Amazon Transcribe pipeline: S3 audio upload → Transcribe → feed transcript to agent
- ☐ Amazon Polly: text-to-speech for agent responses

Documentation (All Members)

- ☐ **README.md** — project overview, team members, quick start

- ☐ **docs/setup.md** — full reproduction steps (clone → provision → deploy → test → teardown)
- ☐ **docs/architecture.md** — at least one architecture diagram:
 - System-level diagram showing all services and data flow
 - Deployment pipeline diagram
 - (Bonus) C4 diagrams, sequence diagrams
- ☐ **docs/llm-usage/** — each member documents:
 - Which LLM tools they used
 - Specific examples of code generation, debugging, architecture planning
 - Critical evaluation: what they accepted, rejected, or modified
 - (Bonus) comparison of LLM-generated vs hand-written code

Presentation Prep

- ☐ Assign presentation sections — each member covers their area:
 1. **Business Problem & Architecture Overview** — any member (2 min)
 2. **Infrastructure & Terraform** — George (3 min)
 3. **CI/CD & Git Workflow** — Troy (3 min)
 4. **ML Endpoints & Data Pipeline** — Kathleen & Okino (3 min)
 5. **Agentic Layer, Bedrock, K8s & Frontend** — George (3 min)
 6. **Live Demo** — all (3 min)
 7. **LLM Usage & Lessons Learned** — all (2 min)
- ☐ Rehearse at least once as a team
- ☐ Prepare for Q&A on any section

Final Checks

- ☐ GitHub Projects board is up to date with task history
- ☐ All PRs have descriptions and at least one review
- ☐ Commit history shows contributions from all members
- ☐ No AWS credentials in the repo (scan with `git log --all -p | grep -i "AKIA"`)
- ☐ All services healthy on EKS
- ☐ CI/CD pipelines green

Milestone ✓

Demo-ready. Clone → follow docs → full deployment reproducible.

Parallel Work Strategy

The key to speed: **everyone works simultaneously from Day 1.**

	Troy	Kathleen/Okino	George
Week 1 Focus:	CI/CD Repo Workflows	ML Models Datasets SageMaker Train FastAPI Wrappers	Infra + Agent Terraform K8s Manifests LangChain

Week 2 Focus:

Frontend
Final CI
Docs

Frontend
Hardening
Docs

Frontend
Deploy to EKS
Docs

Integration points (where people need to sync):

1. **API contracts** — agree on request/response shapes before coding (Day 1)
2. **Docker image names** — Troy needs image paths for CI, George needs them for K8s
3. **Endpoint URLs** — Kathleen/Okino provide SageMaker endpoint names, George puts them in ConfigMaps
4. **Secrets** — Troy sets up GH Secrets, George references them in K8s Secrets

Bonus Points Checklist

Area	Bonus Item	Difficulty	Owner
Containers	Alpine/slim bases, health checks, <code>.dockerignore</code>	Low	All
Terraform	Remote state with S3/DynamoDB	Low	George
Terraform	Modular structure (separate modules)	Medium	George
CI/CD	Separate CI workflows per service	Low	Troy (already in plan)
CI/CD	Branch targeting, rollback, multi-workflow chaining	Medium	Troy
LangChain	LangGraph workflows	Medium	George
LangChain	LangSmith observability	Medium	George
Bedrock/Chat	Conversation memory persistence	Medium	George + frontend dev
Bedrock/Chat	Polished UI (Tailwind/MUI)	Medium	Frontend dev
SageMaker	Third endpoint or model versioning	Medium	Kathleen/Okino
SageMaker	Retry logic, error handling, fallback	Medium	Kathleen/Okino
K8s	ResourceQuota + LimitRange	Low	George
K8s	HPA, persistent storage, rolling updates	Medium	George
Collaboration	Sprint artifacts, retro notes, standup docs	Low	Troy + All
Collaboration	Code review comments on PRs	Low	All
Presentation	C4/sequence diagrams	Medium	All

Area	Bonus Item	Difficulty	Owner
Presentation	Present early	Low	All
LLM Usage	Compare LLM vs hand-written code	Medium	All
LLM Usage	Document prompt engineering techniques	Low	All
Extra	AWS Transcribe/Polly integration	Medium	Anyone
Extra	Blog articles, MkDocs, video series	High	Anyone
Extra	Portfolio page integration	High	Anyone

Daily Standup Template

```

**Name:**
**Yesterday:** What I completed
**Today:** What I'm working on
**Blockers:** Anything stopping progress
**LLM Note:** How I used LLM tools today (for documentation)

```

Key Principles

1. **Vertical slice first** — get one path working end-to-end before expanding
2. **Parallel lanes** — infrastructure and application work happen simultaneously
3. **API contracts on Day 1** — agree on request/response shapes so everyone can work independently
4. **Feature branches always** — never commit directly to **main**
5. **Document as you go** — don't leave it all for Phase 3
6. **Log your LLM usage** — it's 10% of the grade, treat it seriously
7. **Test locally before deploying** — Docker Compose and Minikube first, EKS second
8. **Reuse prior work** — FastAPI services, K8s manifests, Terraform configs from earlier modules are fair game